

JavaScript: arrays e métodos de manipulação

for...of • inserção • remoção • busca • recorte • cópia • ordenação básica

Prof. Dionizio Porfírio
Unichristus • 2026.2

Objetivos de aprendizagem

Ao final da aula, o estudante deverá ser capaz de:

- Percorrer arrays utilizando for e for...of.
- Inserir e remover elementos em diferentes posições de um array.
- Localizar valores e posições utilizando métodos de busca.
- Extrair trechos e combinar arrays sem alterar necessariamente o original.
- Diferenciar métodos que modificam o próprio array de métodos que produzem outro resultado.
- Resolver problemas combinando arrays, repetição, condição, contadores e acumuladores.

Arrays em JavaScript

A ideia de coleção indexada já é conhecida do Java; o comportamento da estrutura é mais flexível no JavaScript.

No Java

O tamanho do vetor é definido na criação e não muda.

Todos os elementos pertencem ao mesmo tipo declarado.

Exemplo: `double[] notas = new double[5];`

No JavaScript

O array pode crescer ou diminuir durante a execução.

A linguagem permite valores de tipos diferentes, embora em algoritmos normalmente agrupemos dados relacionados.

Exemplo: `const notas = [7.5, 8, 6.5];`

Os índices continuam começando em 0 e a propriedade `length` informa a quantidade atual de elementos.

Criação e inicialização de arrays

As formas abaixo produzem arrays, mas não possuem exatamente o mesmo significado.

[]

Forma literal. É a mais direta para criar um array vazio ou já

```
const nomes = [];
```

[valores]

Cria o array já com os elementos informados.

```
const notas = [7, 8, 9];
```

new Array(n)

Com um único número, define o length e deixa as posições vazias.

```
const dados = new Array(5);
```

Array.of(...)

Cria o array com os valores informados. Array.of(5) resulta em

```
const dados = Array.of(5);
```

Array.from(...)

Cria um array a partir de um valor iterável, como uma string.

```
const letras = Array.from("JS");
```

Uso de const

const impede a variável de apontar para outro array, mas não impede alterar seus elementos.

```
const notas = [7, 8];  
notas[0] = 9; // válido
```

Percorrendo um array

O for tradicional é útil quando o índice faz parte da lógica. O for...of entrega diretamente cada valor armazenado.

Acesso pelo índice

```
const notas = [7.5, 8, 6.5, 9];

for (let i = 0; i < notas.length; i++) {
  console.log(i, notas[i]);
}
```

Use for

quando precisar da posição, percorrer apenas parte do array ou controlar o passo.

Acesso direto aos valores

```
const notas = [7.5, 8, 6.5, 9];

for (const nota of notas) {
  console.log(nota);
}
```

Use for...of

quando a lógica depende principalmente dos valores e todos os elementos serão percorridos.

Exemplo — análise de preços

Uma loja deseja analisar os preços de seis produtos cadastrados pelo usuário.

- Leia os seis preços com prompt e armazene-os em um array.
- Percorra o array para calcular o valor total e a média dos preços.
- Em uma nova repetição, conte quantos preços estão acima da média.
- Mostre o array preenchido, a média e a quantidade encontrada.

O exercício retoma for...of, acumulador, contador e conversão da entrada com Number().

Inserção de elementos

Os métodos `push()` e `unshift()` aumentam o array e modificam o próprio array.

`push(valor)`

Insere um ou mais elementos no final. O retorno do método é o novo `length` do array.

```
const fila = ["Ana", "Bia"];
fila.push("Caio");
// ["Ana", "Bia", "Caio"]
```

`unshift(valor)`

Insere um ou mais elementos no início. Os índices dos elementos anteriores são deslocados.

```
const fila = ["Ana", "Bia"];
fila.unshift("Urgente");
// ["Urgente", "Ana", "Bia"]
```

Em Java, aumentar um vetor exige criar outra estrutura; no JavaScript, o array pode mudar de tamanho durante a execução.

Remoção das extremidades

pop() e shift() removem um elemento e devolvem o valor que foi retirado.

pop()

Remove o último elemento do array. É útil quando o fim da estrutura representa o próximo item a ser retirado.

```
const pilha = [10, 20, 30];  
const removido = pilha.pop();  
// removido = 30
```

shift()

Remove o primeiro elemento. Os demais elementos mudam de índice depois da remoção.

```
const fila = ["Ana", "Bia", "Caio"];  
const atendido = fila.shift();  
// atendido = "Ana"
```

Atenção

Se o array estiver vazio, pop() e shift() retornam undefined.

Exemplo — fila de atendimento

Considere a fila inicial: ["Ana", "Bruno", "Carla"]. Atualize a fila conforme os eventos abaixo.

- Diego chega para atendimento comum e deve entrar no final da fila.
- Eva possui prioridade e deve entrar no início da fila.
- O primeiro cliente da fila é chamado para atendimento e deve ser removido.
- Mostre quem foi chamado e como ficou a fila após as operações.

A solução deve utilizar os métodos adequados de inserção e remoção, preservando a ordem da fila.

Alterações no meio do array com splice()

splice() modifica o próprio array e permite remover, inserir ou substituir elementos a partir de uma posição.

```
array.splice(indiceInicial, quantidadeRemover, novosElementos...)
```

1. índice inicial

posição a partir da qual a alteração será realizada.

2. quantidade

quantos elementos serão removidos a partir dessa posição.

3. novos elementos

valores opcionais que serão inseridos no lugar indicado.

O retorno de splice() é um novo array contendo os elementos que foram removidos.

Exemplos com splice()

Partindo do array `const letras = ["A", "B", "C", "D"]`:

Remover

Retire um elemento a partir do índice 1.

```
letras.splice(1, 1);  
// ["A", "C", "D"]
```

Inserir

Não remova elementos e insira "X" antes da posição 2.

```
letras.splice(2, 0, "X");  
// ["A", "B", "X", "C", "D"]
```

Substituir

Remova um elemento e coloque outro valor na mesma posição.

```
letras.splice(2, 1, "Y");  
// ["A", "B", "Y", "D"]
```

Como o método altera o array original, é importante conhecer o estado da estrutura antes e depois da operação.

Extração de trechos com slice()

slice() cria um novo array com parte dos elementos e não modifica o array original.

```
const notas = [5, 6, 7, 8, 9];  
  
const trecho = notas.slice(1, 4);  
  
console.log(trecho);  
// [6, 7, 8]
```

Índices negativos

slice(-3) seleciona os três últimos elementos do array.

Intervalo utilizado

O primeiro índice é incluído. O índice final não é incluído no resultado.

Sem argumentos

notas.slice() produz uma cópia superficial contendo todos os elementos do array.

Diferença entre slice() e splice()

slice()

- produz um novo array;
- não altera o array original;
- seleciona um intervalo;
- o índice final é exclusivo.

Exemplo: dados.slice(1, 4)

splice()

- modifica o array original;
- pode remover, inserir e substituir;
- recebe a quantidade que será removida;
- devolve os elementos removidos.

Exemplo: dados.splice(1, 2)

Os nomes são parecidos, mas os efeitos sobre o array são diferentes.

Verificando a presença de um valor

`includes()` responde apenas se o valor está ou não presente no array.

```
const cursos = ["CC", "SI", "ADS"];

console.log(cursos.includes("SI"));
// true

console.log(cursos.includes("Engenharia"));
// false
```

Retorno

O resultado é sempre booleano: `true` ou `false`.

Comparação

A busca diferencia valores de tipos diferentes e também diferencia maiúsculas de minúsculas em strings.

Localizando a posição de um valor

`indexOf()` procura a primeira ocorrência. `lastIndexOf()` procura a última ocorrência.

```
const codigos = [10, 20, 30, 20, 40];  
  
console.log(codigos.indexOf(20));  
// 1  
  
console.log(codigos.lastIndexOf(20));  
// 3  
  
console.log(codigos.indexOf(99));  
// -1
```

Valor encontrado

O método devolve o índice correspondente.

Valor ausente

Quando não encontra o valor, o retorno é -1. Esse resultado pode ser usado em uma condição.

```
if (codigos.indexOf(99) !== -1) { ... }
```

Exemplo — lista de presença

Um sistema possui o array ["Ana", "Bruno", "Carla", "Bruno", "Davi"].

- Solicite ao usuário um nome para consulta.
- Informe se o nome aparece na lista.
- Caso apareça, mostre a posição da primeira ocorrência e da última ocorrência.
- Caso não apareça, exiba uma mensagem informando que o nome não foi localizado.

O objetivo é escolher adequadamente entre `includes()`, `indexOf()` e `lastIndexOf()`.

Combinação de arrays

concat() cria um novo array contendo os elementos das estruturas informadas, sem modificar os arrays de origem.

```
const turmaA = ["Ana", "Bia"];  
const turmaB = ["Caio", "Davi"];  
  
const turma = turmaA.concat(turmaB);  
  
console.log(turma);  
// ["Ana", "Bia", "Caio", "Davi"]
```

Resultado

concat() devolve outro array. turmaA e turmaB permanecem como estavam.

Mais de um argumento

Também é possível combinar mais arrays ou adicionar valores individuais na mesma chamada.

Transformando um array em texto com join()

join() concatena os elementos e devolve uma string. O separador é definido pelo argumento do método.

```
const linguagens = ["Java", "JavaScript", "Python"];

console.log(linguagens.join(", "));
// Java, JavaScript, Python

console.log(linguagens.join(" -> "));
// Java -> JavaScript -> Python
```

Resultado

O array não é alterado. O valor devolvido por join() é uma string.

Uso comum

É útil para montar uma saída legível, uma linha de relatório ou um texto contendo os elementos de uma coleção.

Ordenação e inversão

`sort()` reorganiza os elementos; `reverse()` inverte a ordem atual. Os dois métodos modificam o próprio array.

```
const nomes = ["Davi", "Ana", "Carla", "Bruno"];

nomes.sort();
// ["Ana", "Bruno", "Carla", "Davi"]

nomes.reverse();
// ["Davi", "Carla", "Bruno", "Ana"]
```

Strings

Sem uma função de comparação, `sort()` funciona bem para muitos casos de ordenação alfabética simples.

Números

A ordenação numérica exige uma função de comparação. Essa forma será retomada na aula de funções e callbacks.

`[2, 10, 3].sort()` não produz necessariamente a ordem numérica esperada.

Cópia de arrays e referência

Atribuir um array a outra variável não duplica os elementos: as duas variáveis passam a referenciar a mesma estrutura.

```
const original = [10, 20, 30];  
const copia = original;  
  
copia[0] = 99;  
  
console.log(original);  
// [99, 20, 30]
```

Mesma referência

const copia = original não cria outro array.

```
const original = [10, 20, 30];  
const copia = original.slice();  
  
copia[0] = 99;  
  
console.log(original);  
// [10, 20, 30]
```

Nova estrutura

slice() sem argumentos é uma forma simples de criar uma cópia superficial.

Métodos que alteram o array original

Antes de usar um método, é importante saber se a operação muda a estrutura armazenada na variável.

Modificam o próprio array

`push()` adiciona no final
`unshift()` adiciona no início
`pop()` remove do final
`shift()` remove do início
`splice()` altera posições internas
`sort()` reorganiza a ordem
`reverse()` inverte a ordem

Não modificam o array original

`slice()` cria um recorte/cópia
`concat()` combina em novo array
`includes()` retorna true ou false
`indexOf()` retorna uma posição
`lastIndexOf()` retorna uma posição
`join()` retorna uma string

O valor de retorno de cada método também é diferente: novo length, elemento removido, novo array, índice, booleano ou string.

Cuidados ao manipular arrays

new Array(5)

Cria um array com length 5 e posições vazias. Não é o mesmo que [5].

Índice inexistente

A leitura de uma posição fora do intervalo devolve undefined.

delete

delete array[i] deixa uma posição vazia. Para remover e reorganizar índices, prefira splice().

Método x propriedade

length não utiliza parênteses. push(), slice() e outros métodos utilizam.

Método mutável

Depois de splice(), sort() ou shift(), não assuma que o array continua como antes.

Tipo de retorno

Nem todo método retorna um array. indexOf() retorna número; includes(), booleano; join(), string.

Exemplo — controle de tarefas

Um aplicativo mantém uma lista de tarefas pendentes.

- Comece com o array ["estudar", "revisar", "enviar atividade"].
- Adicione uma nova tarefa comum ao final e uma tarefa urgente no início.
- Solicite ao usuário o nome de uma tarefa cancelada. Caso ela exista, remova somente a primeira ocorrência.
- Marque a primeira tarefa como concluída removendo-a da lista.
- Mostre a lista final em uma única linha, separando as tarefas por " | ".

O problema combina inserção, busca de posição, remoção em posição específica, fila e join().

Resumo dos principais métodos

Inserir e remover

push() / pop()
unshift() / shift()
splice()

Buscar

includes()
indexOf()
lastIndexOf()

Recortar e combinar

slice()
concat()
join()

Organizar

sort()
reverse()

Percorrer

for
for...of

Consultar a estrutura

length
array[indice]

A escolha do método depende da operação desejada e de o array original poder ou não ser alterado.

Exercício 1 — fila de chamados

Uma central de suporte mantém, em um array, os códigos dos chamados que aguardam atendimento.

- Leia seis códigos de chamados e armazene-os na ordem em que foram recebidos.
- Depois da leitura, receba um código prioritário e coloque-o antes dos demais.
- Receba também um novo chamado comum e coloque-o após todos os chamados existentes.
- Em seguida, receba o código de um chamado cancelado. Caso ele esteja na fila, remova somente a primeira ocorrência.
- Atenda o primeiro chamado da fila, removendo-o e exibindo seu código.
- Por fim, mostre a fila restante e informe se um código consultado pelo usuário ainda está aguardando atendimento.

Exercício 2 — histórico de preços

Um sistema mantém o histórico diário do preço de um produto.

- Leia sete preços e armazene-os em um array.
- Receba um preço informado como lançamento incorreto. Caso exista, remova apenas sua primeira ocorrência.
- Leia o preço do dia atual e acrescente-o ao final do histórico.
- Calcule e apresente a média dos preços que permaneceram no array.
- Crie um novo array contendo somente os três últimos preços do histórico, sem alterar o array principal.
- Solicite um preço para consulta e informe se ele existe no histórico e, quando existir, a posição de sua primeira ocorrência.
- Apresente o histórico final e o recorte dos três últimos preços.